LR-Agent / Agent 后端入门

第09章
个人改造与简历面试复盘

围绕真实业务、源码与实验建立项目表达，逐步把理解与复现转化为亲自完成、能够追问的后端经历。

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 用一次任务讲清Agent后端的职责、流程与取舍。
- 分清源码事实、实验、已有验收与个人贡献。
- 选择一个可复现的小改造并组织证据。
- 准备简历结构、两分钟项目介绍和失败场景追问。

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

9.1 场景：面试官让你介绍这个项目 . . . 3
9.2 先画职责边界，再列技术栈 . . . 4
9.3 能力清单：哪些已经实现，哪些没有证据 . . . 5
9.4 用一次请求串起核心设计 . . . 6
9.5 完整实验：核对自己准备写进简历的能力 . . . 7
9.6 建立一条可回放的证据链 . . . 8
9.7 个人贡献怎样分层记录 . . . 9
9.8 选择一个可完成的后端改造 . . . 10
9.9 真实案例：日志断管道为什么不该中断服务 . . . 11
9.10 用两分钟讲清问题与设计 . . . 12
9.11 简历条目怎样写得具体 . . . 13
9.12 高频追问：定义之后要有项目定位 . . . 14
9.13 失败场景：设计取舍怎么回答 . . . 15
9.14 完整实验：检查个人改造记录 . . . 16
9.15 综合练习与参考思路 . . . 17
9.16 从教材走向自己的项目经历 . . . 18
9.17 源码图解：重复结果怎样被过滤 . . . 19

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

9.1 场景：面试官让你介绍这个项目

面试官问：“你这个 Agent 项目解决什么问题，你做了哪些工作？”先不要报框架清单。用一个真实输入说明业务，再讲后端决策、工具执行、人工确认及验证证据。

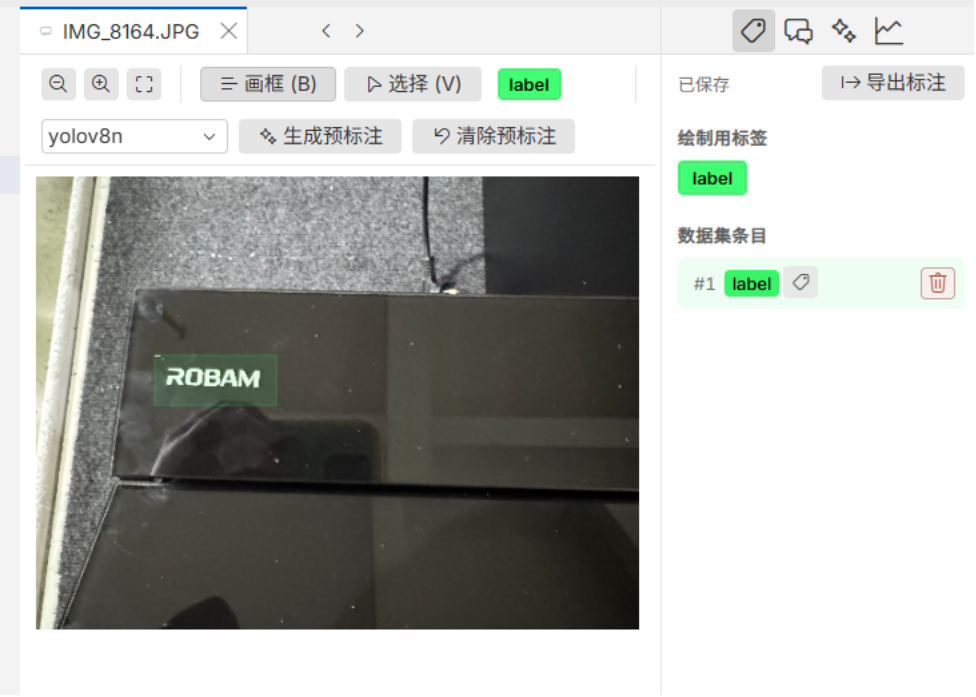


图9-1 2026-10-17已有真实验收画面：确认并重启回读后，图像上仍有已保存矩形框。本轮未重新进行云模型与YOLO验收；截图不证明检测精度。

这张图只帮助理解结果：用户想要的不是一段建议，而是可审核的标注。学习重点仍是后端怎样得到和控制结果，不要求讲解界面实现。

本章目标

把前面学过的模块串成一个可解释、可验证的后端方案，再选择读者亲自完成的小改造。下文参考回答都要结合自己的实际学习记录。

9.2 先画职责边界，再列技术栈

层次	主要职责	讲解时的边界
模型与框架	消息、模型适配、官方工具循环	模型提出调用，程序负责执行与限制
业务约束	阶段、合批、去重、预算与错误	规则不能仅依赖Prompt提醒
服务与协议	请求校验、SSE、任务ID、取消和结果续跑	外部执行方只作为协议参与者
推理工具	实际图像推理、坐标与标签结果	工具集成不等于模型训练
数据与验收	提案、确认、回读及测试记录	图State、业务快照与数据库分开

LangChain 用于统一模型、消息与工具接口；create_agent 承接模型和工具循环。LangGraph 为执行提供图基础，项目再施加业务约束。这些层次需要分别说明，不能一概称为“我写了一个 LangGraph 工作流”。

主修源码位于 LR-Agent/vendor/local-agent。认证后端主要承担账号等职责，不是 Agent 图状态存储。外部调用方和推理执行方只需说明输入输出，不展开前端学习。

9.3 能力清单：哪些已经实现，哪些没有证据

可以核对的能力	对应证据	不能扩大的说法
官方工具循环	runtime.py 的工厂与离线消息实验	不是所有请求都有工具
阶段门控与合批	task_phase.py、round_plan.py、相关测试	不是全局事务或任意并发幂等
请求续跑	resume.py 的消息和结果重建	不是持久化图Checkpoint恢复
只读子代理	assist/explore_readonly.py 的工具白名单	不是完整多Agent协同平台
真实标注闭环	已有两图确认、保存与重启回读记录	不是精度/效率基准

“源码存在”“离线实验通过”“已有真实验收”“尚未验证”是四种证据。面试陈述要明确是哪一种，尤其不能把固定模型结果当作云模型能力。

技术范围

本项目没有新增RAG或向量库。Context、SQLite、Markdown记忆与认证PostgreSQL各有职责，不能为了让技术栈看起来完整而互相替代。

9.4 用一次请求串起核心设计



图9-2 典型外部工具调用的简化时序；不表示所有工具都经过第三方或存在数据库事务。

介绍时先讲正常路径，再补两个分支：有工具需求时走官方循环；纯聊天可以直接返回文本流。外部工具挂起后，下一请求携带结果恢复语义上下文，不是服务器从持久化图中自动找回执行现场。

设计取舍：后端能够集中控制模型与工具规则，但必须明确跨请求数据来源、调用ID与副作用边界。不能把这些工程责任都交给模型回答。

9.5 完整实验：核对自己准备写进简历的能力

如果准备写“采用LangGraph持久化恢复”，先检查项目工厂到底传了什么。下面通过当前源码做一个窄范围事实检查。

完整教学实验 ch09_factory_evidence。在 study 自行新建 my_ch09_factory_evidence.py，复制全部代码。

```python
import ast
import inspect
from app.agent.langchain_runtime.runtime import build_langchain_agent

tree = ast.parse(inspect.getsource(build_langchain_agent))
calls = [node for node in ast.walk(tree)
         if isinstance(node, ast.Call)
         and isinstance(node.func, ast.Name)
         and node.func.id == "create_agent"]
assert len(calls) == 1
names = {item.arg for item in calls[0].keywords}
print("context schema:", "context_schema" in names)
print("middleware:", "middleware" in names)
print("explicit checkpointer:", "checkpointer" in names)
print("explicit store:", "store" in names)
```

运行：& $agentPy ".\study\my_ch09_factory_evidence.py"（工作区根目录）

本轮实际输出：

```
context schema: True
middleware: True
explicit checkpointer: False
explicit store: False
```

inspect.getsource 读取当前函数的源码，ast.parse 把它转为语法树；这里检查的是 create_agent 调用显式传入的参数名，不请求模型。

输出证明当前工厂传入 Context 和 middleware，未显式配置图 checkpointer 或 Store。它不是压力、恢复或数据持久性测试。面试前还应结合入口和 resume.py 解释实际续跑。

9.6 建立一条可回放的证据链



图9-3 验证记录组织方式；这是方法图，不是新增执行流程。

例如已有验收只覆盖两张图、每张一个bbox及确认回读。它能支持“这条具体路径跑通”的结论，不能推出任意标注类型或任意机器都可靠。

记录字段	填写示例	注意事项
输入与条件	两张图片副本、追加模式	保留数据范围与版本
检查方式	事件合同 + 文件回读	截图只能作为一部分证据
结论边界	当前环境中功能成功	不补写未经测量的性能数字

9.7 个人贡献怎样分层记录

层级	需要留下什么	适合的表达
阅读理解	函数位置、调用链、自己的解释	分析并理解某模块设计
亲自复现	命令、输入、输出和失败条件	本地复现并验证某行为
亲自改造	本人提交、前后行为、回归结果	针对某问题实现具体改进

本轮教材与日志修复属于协作产物。读者在面试前需要亲自运行、修改和解释相关代码；不能仅因材料已经制作完成，就说自己独立实现了全部功能。

每学一章可以写四句话：我原来不懂什么；哪个实验帮助我理解；我具体改了什么；哪些结论还不能证明。它们比背诵一段宏观架构介绍更能支持追问。

现有测试数量的表达

原有568项后端测试通过不等于本人新增568项。新改造新增了多少条测试，应从实际差异与执行结果中统计。

9.8 选择一个可完成的后端改造

建议把范围限定为一个明确失败输入，而不是“升级成多Agent平台”。例如：对重复结果补一条合同测试，或改进某类工具错误的反馈，再核对正常分支是否受影响。



图9-4 个人改造练习的工作路径；不是已经完成的功能清单。

选题例子	验收方法	完成后才可表达
重复结果合同	相同ID再次回传，观察消息与执行次数	明确范围内防重复验证
工具错误反馈	固定失败输入与后续模型行为	错误语义与恢复路径改进
预算边界测试	限制前后调用次数和结束消息	边界条件覆盖

9.9 真实案例：日志断管道为什么不该中断服务

本轮实际遇到 EPIPE: broken pipe, write。栈对应子进程日志回调中的console.log：Python产生输出后，外壳尝试写入已关闭的日志管道，造成未捕获错误。这不是图像推理算法失败。



图9-5 本轮实际故障处理的简化流程；不要求学习桌面前端实现。

新增守卫测试11项通过，其中stdout、stderr两个真实OS断管道场景均验证后续任务经独立IPC返回结果5，正常退出0。这个5只是测试计算结果，不是性能指标。

如何用于面试

可学习“按栈定位 → 区分故障层 → 最小修复 → 故障注入”的方法。只有自己能够复现并解释时，才将它写入个人维护经历。详见日志断管道修复记录。

9.10 用两分钟讲清问题与设计

顺序	回答要点	避免空泛
问题	自然语言任务需要落到可审核的标注操作	先举一个输入
方案	统一模型/工具接口 + 官方循环 + 业务约束	解释一个具体约束
取舍	模型负责选择，代码负责执行边界与确认状态	说明为什么不用纯Prompt
证据	一组离线实验及范围明确的真实验收	说清谁做的、测了什么
限制	图持久化、并发、精度等未验证部分	给出下一步验证方法

参考结构：“基于现有LR-Agent，我关注自然语言到工具执行的后端链路。项目使用LangChain统一模型与工具，通过create_agent组织循环，并用阶段和回合约束避免未确认就继续写入。我的具体工作是【本人实际改造】；我用【实际测试】验证【行为】，尚未覆盖【限制】。”

括号部分必须由个人证据填写。若当前只完成学习，直接说明阅读和复现范围，也可以准确展示理解深度。

9.11 简历条目怎样写得具体

条目部分	填写内容	证据位置
业务背景	面向工作区标注与人工审核	真实任务输入与闭环图
本人职责	具体后端模块或小改造	提交与改动文件
实现方式	工具Schema、约束、续跑或测试	函数与数据流
验证结果	功能断言、失败恢复与范围	运行记录、测试输出

学习阶段可以写项目笔记：“分析LangChain Agent工具循环和请求续跑，运行离线实验验证消息关联与阶段约束。”这描述理解与复现，不声称从零开发。

完成个人改造后可改为：“针对【明确触发条件】导致【问题】，在【模块】实现【改动】，通过【实际用例与断言】验证【前后行为】。”是否写数字取决于是否有真实测量。

不要补造结果

没有人工耗时对照就不写效率提升；没有标注集评价就不写准确率；没有并发测试就不写高并发能力。框架名字不能代替具体工程工作。

9.12 高频追问：定义之后要有项目定位

追问	回答主线	要能指到哪里
Chain和Agent区别?	固定步骤与工具决策循环；讨论成本和可控性	runtime.py、最小工具实验
Context和State区别?	调用注入信息与执行中更新信息；再问保留多久	AgentContext、BusinessRoundState
为什么要Middleware?	将阶段、预算和错误约束放到明确执行钩子	middleware.py、round_plan.py
怎样续跑?	历史消息、工具ID与本次结果重建；没有图checkpoint	resume.py
MCP和子代理有何不同?	协议与委派能力不同；说明各自限制	app/agent/tools/mcp_client.py、subagent.py

30秒回答适合先给定义；随后补一条真实输入、一个函数和一个测试。面对没验证过的行为，应说明当前证据和准备怎样验证，不用猜测一个笼统保证。

例如被问“既然有completed_tool_ids，是不是任意并发都不会重复？”应先解释其请求级来源与检查位置，再说明并发副作用需要独立验证，不能把集合去重扩大成分布式保证。

9.13 失败场景：设计取舍怎么回答

场景	先定位的问题	合适的验证
模型重复发工具	ID、回合规划与执行范围	固定消息 + 记录执行次数
外部工具执行失败	错误结果如何关联与返回	失败输入与ToolMessage合同
断流或取消	取消检查点、结束事件与已发生副作用	中断阶段分别注入
视觉读取失败	主/辅助模型能力与工具降级路径	可控错误与回退测试
服务重启后续跑	请求是否完整，状态保存在哪里	重建输入及存储边界核对

讨论取舍时同时说收益与代价。使用统一框架减少手写循环，但仍要处理版本与事件适配；请求重建减少服务器跨请求状态，但依赖调用方提供一致历史；只读子代理限制副作用，也限制可委派任务。

取消不是回滚

已发生的文件或推理副作用不因停止后续生成而自动撤销。业务确认、补偿和快照属于另外的机制；回答时先说清实际实现。

9.14 完整实验：检查个人改造记录

把贡献描述转成可检查字段，帮助区分“学过”和“改过”。下面仅演示记录组织，不是自动生成简历或替人认定贡献。

完整教学实验 ch09_contribution_record。在 study 自行新建 my_ch09_contribution_record.py，复制全部代码。

```python
def complete_change_record(record: dict) -> bool:
    return (record["level"] == "change"
            and bool(record.get("commit"))
            and record["verified"])

records = [
    {"topic": "read loop", "level": "reading", "verified": True},
    {"topic": "run sample", "level": "reproduce", "verified": True},
    {"topic": "own fix", "level": "change",
     "commit": "teaching-only", "verified": True},
]
for record in records:
    print(record["topic"], "->", complete_change_record(record))
```

运行：& $agentPy ".\study\my_ch09_contribution_record.py"（工作区根目录）

本轮实际输出：

```
read loop -> False
run sample -> False
own fix -> True
```

这是教学记录示例。字典中的 level 区分阅读、复现与修改；bool 和 and 组合检查是否留下提交及验证记录。teaching-only 是假标记，不是项目真实提交。

返回 True 只表示这个教学记录字段齐全，不会自动判断个人贡献质量，更不会证明该修改已由读者完成。真实材料还要有修改内容、失败案例及可复现命令。

9.15 综合练习与参考思路

练习一：画出一次模型发起外部工具调用后，请求在哪里结束、结果怎样回传。参考：沿入口、官方循环、中间件与SSE适配找到tool_pending，再沿client_tool_results进入resume重建；用ID关联，不假设checkpointer。

练习二：选择一个当前门控用例，改变提案状态或路径，预测结果再运行。参考：先区分await_confirm与verify，再看工具名和路径条件；不把纯函数实验当作真实落盘。

练习三：把“我做了一个高性能多Agent标注系统”改成可验证描述。参考：先删除未测性能与未实现的多Agent编排，写具体工具循环、业务约束及本人实际改造和测试。

验收维度	通过条件	你应保存的材料
概念	能解释三个易混点	自己的短文或图
源码	能定位一个正常与失败分支	路径、符号与输入输出
实验	能修改输入并解释结果	命令、输出、断言
面试	能讲一个取舍与未覆盖边界	两分钟复盘与证据索引

9.16 从教材走向自己的项目经历



图9-6 后续学习路径，不代表读者已经完成所有步骤。

本套材料的主线是Agent后端：模型与消息、工具循环、业务约束、接口续跑、扩展边界及测试。图表和截图帮助定位事实，最终还需要你亲自运行、修改和复述。

最后的自测

不看教材，说明一个输入经过哪些函数、产生哪些消息、哪里限制副作用，以及你怎样证明结果。能把这些问题回答具体，再把对应内容写进简历。

9.17 源码图解：重复结果怎样被过滤

本页只读Python后端。prepare_resume_messages先复制历史，再过滤已有结果的工具ID；没有新结果时直接返回，否则补齐消息。结合第5章实验理解它的请求范围，不扩大为全局事务保证。

真实源码阅读图 / 续跑前过滤重复结果

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/resume.py

```
134 def prepare_resume_messages(
135     lc_messages: Sequence[Any],
136     client_tool_results: Sequence[Any] | None = None,
137     *,
138     user_content: str = "",
139     phase_ctx: TaskPhaseContext | None = None,
140 ) -> list:
... # 文档说明省略，以下保留原始行号
149     messages = list(lc_messages)
150     fresh = unanswered_client_tool_results(messages, client_tool_results)
151     if not fresh:
152         return messages
153     return append_client_tool_results_to_messages(
154         messages,
155         list(fresh),
156         user_content=user_content,
157         proposals_applied=proposals_applied(phase_ctx),
158     )
```

真实Python后端源码阅读图：prepare_resume_messages，文档说明省略；保留原始行号和完整处理主体。

先看什么	怎样读	自己记录什么
输入	历史消息、本次工具结果与阶段	哪些参数允许None，哪些有默认值
处理	复制列表、过滤已回答ID，再补消息	fresh为空时为何直接返回
输出	返回新消息列表，不修改输入列表	重复回传是否增加ToolMessage

源码与证据索引

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::build_langchain_agent

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/resume.py

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py

docs/2026-10-17本地主流程验收记录.md

docs/2026-10-17日志断管道修复记录.md

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。